

Java String

In **Java**, string is basically an object that represents sequence of char values.

An **array** of characters works same as Java string. For example:

1. **char**[] ch={'j','a','v','a','t','p','o','i','n','t'};
2. String s=**new** String(ch);

is same as:

1. String s="javatpoint";

What is String in java

Generally, String is a sequence of characters.

But in Java, string is an object that represents a sequence of characters.

The `java.lang.String` class is used to create a string object.

How to create a string object?

There are two ways to create String object:

1. By string literal
2. By new keyword

1) String Literal

Java String literal is created by using double quotes. For Example:

1. String s="welcome";

Each time you create a string literal, the JVM checks the "string constant pool" first. If the string already exists in the pool, a reference to the pooled instance is returned.

If the string doesn't exist in the pool, a new string instance is created and placed in the pool.

For example:

1. String s1="Welcome";
2. String s2="Welcome";//It doesn't create a new instance

2) By new keyword

1. `String s=new String("Welcome");`//creates two objects and one reference variable

In such case, JVM will create a new string object in normal (non-pool) heap memory, and the literal "Welcome" will be placed in the string constant pool.

The variable s will refer to the object in a heap (non-pool).

```
public class StringExample
{
    public static void main(String args[])
    {
        String s1="java";//creating string by java string literal
        char ch[]={'s','t','r','i','n','g','s'};
        String s2=new String(ch);//converting char array to string
        String s3=new String("example");//creating java string by new keyword
        System.out.println(s1);
        System.out.println(s2);
        System.out.println(s3);
    }
}
```

No.	Method	Description
1	char charAt(int index)	returns char value for the particular index
2	int length()	returns string length
3	static String format(String format, Object... args)	returns a formatted string.
4	static String format(Locale l, String format, Object... args)	returns formatted string with given locale.
5	String substring(int beginIndex)	returns substring for given begin index.

6	String substring(int beginIndex, int endIndex)	returns substring for given begin index and end index.
7	boolean contains(CharSequence s)	returns true or false after matching the sequence of char value.
8	static String join(CharSequence delimiter, CharSequence... elements)	returns a joined string.
9	static String join(CharSequence delimiter, Iterable<? extends CharSequence> elements)	returns a joined string.
10	boolean equals(Object another)	checks the equality of string with the given object.
11	boolean isEmpty()	checks if string is empty.
12	String concat(String str)	concatenates the specified string.
13	String replace(char old, char new)	replaces all occurrences of the specified char value.
		AMAN DEEP SINGH

14	String replace(CharSequence old, CharSequence new)	replaces all occurrences of the specified CharSequence.
15	static String equalsIgnoreCase(String another)	compares another string. It doesn't check case.
16	String[] split(String regex)	returns a split string matching regex.
17	String[] split(String regex, int limit)	returns a split string matching regex and limit.
18	String intern()	returns an interned string.
19	int indexOf(int ch)	returns the specified char value index.
20	int indexOf(int ch, int fromIndex)	returns the specified char value index starting with given index.

21	int indexOf(String substring)	returns the specified substring index.
22	int indexOf(String substring, int fromIndex)	returns the specified substring index starting with given index.
23	String toLowerCase()	returns a string in lowercase.
24	String toLowerCase(Locale l)	returns a string in lowercase using specified locale.
25	String toUpperCase()	returns a string in uppercase.
26	String toUpperCase(Locale l)	returns a string in uppercase using specified locale.
27	String trim()	removes beginning and ending spaces of this string.
28	static String valueOf(int value)	converts given type into string. It is an overloaded method.

```
class Testimmutablestring
```

```
{
```

```
    public static void main(String args[])
```

```
{
```

```
    String s="Sachin";
```

```
    s.concat(" Tendulkar");//concat() method appends the string at the end
```

```
    System.out.println(s);//will print Sachin because strings are immutable objects
```

```
}
```

```
}
```

Output: Sachin

String compare by equals() method

The `String equals()` method compares the original content of the string. It compares values of string for equality.

`String` class provides two methods:

- `public boolean equals(Object another)` compares this string to the specified object.
- `public boolean equalsIgnoreCase(String another)` compares this `String` to another string, ignoring case.

Hello 1

HELLO 2

hello 3

```
class Teststringcomparison1
```

```
{  
    public static void main(String args[])  
    {  
        String s1="Sachin";  
        String s2="Sachin";  
        String s3=new String("Sachin");  
        String s4="Saurav";  
        System.out.println(s1.equals(s2)); //true  
        System.out.println(s1.equals(s3)); //true  
        System.out.println(s1.equals(s4)); //false  
    }  
}
```

```
class Teststringcomparison1
```

```
{  
    public static void main(String args[])  
    {  
        String s1="Sachin";  
        String s2="sachin";  
        String s3=new String("Sachin");  
        String s4="Saurav";  
        System.out.println(s1.equalsIgnoreCase(s2)); //true  
        System.out.println(s1.equals(s2)); //false  
        System.out.println(s1.equalsIgnoreCase(s3)); //true  
        System.out.println(s1.equalsIgnoreCase(s4)); //false  
    }  
}
```

String compare by compareTo() method

The `String compareTo()` method compares values lexicographically and returns an integer value that describes if first string is less than, equal to or greater than second string.

Suppose `s1` and `s2` are two string variables. If:

- `s1 == s2` :0
- `s1 > s2` :positive value
- `s1 < s2` :negative value

```
class Teststringcomparison4
```

```
{  
    public static void main(String args[])  
    {  
        String s1="Sachin";  
        String s2="Sachin";  
        String s3="Ratan";  
        System.out.println(s1.compareTo(s2)); //0  
        System.out.println(s1.compareTo(s3)); //1(because s1>s3)  
        System.out.println(s3.compareTo(s1)); //-1(because s3 < s1 )  
    }  
}
```

Output:

0
1
-1

Sachin

Sachon

Sachin 6

compareToIgnoreCase()

String Concatenation by concat() method

The String concat() method concatenates the specified string to the end of current string. Syntax:

```
public String concat(String another)
```

Example

```
class TestStringConcatenation3
```

```
{  
    public static void main(String args[])  
    {  
        String s1="Sachin ";  
        String s2="Tendulkar";  
        String s3=s1.concat(s2);  
        // String s3=s2.concat(s1); // Tendulkar Sachin  
        System.out.println(s3); //Sachin Tendulkar  
    }  
}
```

OUTPUT :

Sachin Tendulkar

Java String toLowerCase()

The java string toLowerCase() method returns the string in lowercase letter.

In other words, it converts all characters of the string into lower case letter.

The toLowerCase() method works same as toLowerCase(Locale.getDefault()) method.

It internally uses the default locale.


```
public class StringLowerExample
{
    public static void main(String args[])
    {
        String s1="JAVACLASS stARTING";
        String s1lower=s1.toLowerCase();
        // String s1lower=s1.toUpperCase(); // Make all the character upper case
        System.out.println(s1lower);
    }
}
```

OUTPUT:

javaclass starting

trim() method

The string trim() method eliminates white spaces before and after string.

```
String s=" Sachin ";
```

```
System.out.println(s);
```

```
System.out.println(s.trim());
```

Sachin

Sachin

startsWith() and endsWith() method

```
String s="Sachin";
```

```
System.out.println(s.startsWith("Sa"));
```

```
System.out.println(s.endsWith("n"));
```

```
true
```

```
true
```

charAt() method

The string charAt() method returns a character at specified index.

```
String s="Sachin";
```

```
System.out.println(s.charAt(0));
```

```
System.out.println(s.charAt(3));
```

S

h

length() method

The string length() method returns length of the string.

1. String s="Sachin";
2. System.out.println(s.length()); //6